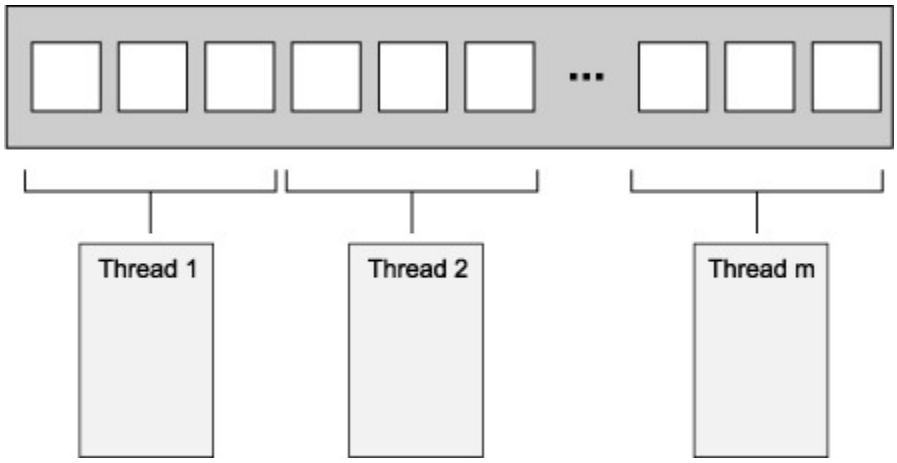
Figure 8.1. Distributing consecutive chunks of data between threads



This structure will be familiar to anyone who has programmed using the Message Passing Interface (MPI)[1] or OpenMP[2] frameworks: a task is split into a set of parallel tasks, the worker threads run these tasks independently, and the results are combined in a final *reduction* step. It's the approach used by the `accumulate` example from section 2.4; in this case, both the parallel tasks and the final reduction step are accumulations. For a simple `for_each`, the final step is a no-op because there are no results to reduce.

[1] http://www.mpi-forum.org/

[2] http://www.openmp.org/

Identifying this final step as a reduction is important; a naïve implementation such as listing 2.8 will perform this reduction as a final serial step. However, this step can often be parallelized as well; `accumulate` actually *is* a reduction operation itself, so listing 2.8 could be modified to call itself recursively where the number of threads is larger than the minimum number of items to process on a thread, for example. Alternatively, the worker threads could be made to perform some of the reduction steps as each one completes its task, rather than spawning new threads each time.

Although this technique is powerful, it can't be applied to everything. Sometimes the data can't be divided neatly up front because the necessary